8*1/15 = 9/15

INDIAN INSTITUTE OF TECHNOLOGY MADRAS

B

Roll No. | 1 | D | 2 | 5 | S | 0 | 2 | 7 |

Name : PUNEET

Total No. of Pages

Quiz I ☐ Quiz II/ Mid-Sem ☐ End-Semester ☐ Make-up ☐ Date : 4/11/2026

Semester & Degree : II, MS(R) Course No. MA5105 Part :

Question No.	1	2	3	4	5	6	7	8	9	10
Marks										

11	12	13	14	15	16	17	18	19	20	Total

Answer on both sides of the paper including the space below

1. (a) For each variable, we will store the value and its derivatives, like in a pair → so for this we can use a tuple.

eg (a, ȧ) → derivatives
↳ numerical value

The arithmetic operations will be handled with the help of defined functions.

Same thing, once we have the basic arithmetic operation. We can easily define f(x) return that function. Now that function is just made up of simple arithmetic operations, we will just call it.

```
class AD:
    def __init__(self, a, b, ȧ, ḃ)
        a = self.a
        b = self.b
        ȧ = self.ȧ
        ḃ = self.ḃ

    def add(self, a, ȧ, b, ḃ)
        return (a+b, ȧ + ḃ)
```

Note: There might be one more way where in, let's say you are only given a & b, then we can have seperate func^n which calculates the derivative of a, then next func^n which forms a tuple pair $(a, \dot{a})$.
In this scenario, I am assuming that $a, \dot{a}$ are given to them how will I take things forward.

Just like add function, we will have subtract, multiplication, division, etc.

Now how to solve the elementary mathematical functions

```
def g(a, ȧ):
    return (g(a), g'(a) ȧ)
```

Obs. we will have to define what $g(a)$ is, then we can just derive $g'(a)$

(b) 2 variables → a, b

their derivatives → $\dot{a}$, $\dot{b}$

their pairs will look like

$(a, \dot{a})$ & $(b, \dot{b})$

then Multiplication –

```
def multiply ((a, ȧ), (b, ḃ)):
    return (ab, ab + bȧ)
def div ((a, ȧ), (b, ḃ)):
    return (a/b, (bȧ - aḃ)/b²)
def exp (a, ȧ)
    return (e^a, e^a ȧ)
def sine (a, ȧ)
    return (sine(a), siné(a) ȧ)
```

(C) So we have 3 variables

```
def f(x, y, z):
    return ( e^-(x² + 2y² + z²) +
             sin(xy) +
             cos(yz) +
             log(1 + x² + y² + z²)
```

Now we know

(xy) is nothing but

$((x, \dot{x}), (y, \dot{y}))$

then we call my

def multiply for earlier

that will give resultant

product for this.

Whatever I get from there

will use sine $(a, \dot{a})$, given

me its resultant operation.

Like wise, we can get the

result for $\cos(yz)$

In this way, we can solve

all 4 components of e, sine, cos,

and log.

Now for e,

we know

def exp $(a, \dot{a})$

return $(e^a, e^a \dot{a})$

but we have

$-(x^2 + 2y^2 + z^2)$

which now I can use add

function

def add $(x, \dot{x}, y, \dot{y}, z, \dot{z})$

return $(x + y + z, \dot{x} + \dot{y} + \dot{z})$

but we also before add component like x^2, y^2, z^2

so for this I will use multiply but with self.

(8) like def multiply (x, $\dot{x}$):
return ($x \times x$, $x \times \dot{x} + x \times \dot{x}$)

likewise this will be a generic function that gets applied to x, y, z.

Finally, we have log
it has $(1 + x^2 + y^2 + z^2)$
so my add function resolves this
internally it has things like x^2, y^2, z^2
so multiply with self again works for me.

So I one go when I run it 4 components

$e^{-(x^2 + 2y^2 + z^2)}$ → call def exp → call def add → call multiply

current back

$\sin(xy)$ or $\cos(xy)$ → call def sin or def cos → call multiply

current back

finally,

$\log(1 + x^2 + y^2 + z^2)$ → call dflog → call def add → call def multiply

← revert back.

(d) Hessian Matrix

$$\begin{bmatrix} a^2 & ab \\ ba & b^2 \end{bmatrix}$$ it is something like this.

so essentially we need to have a vector or vector to form a matrix.

For all the diagonal entries, we have multiply with self & get the final answer.

Whereas for other entries we have calculate multiplication, with 2 variables.

We just need the positional info. using 2 for loop & store the value & its derivative information.